Front-End & SPA - React

Concepts de base du Web Front-End

Rappels des fondamentaux des sites web statiques

- **Définition** : Un site web statique est composé de fichiers HTML séparés pour chaque page.
- **Fonctionnement :**
  - Lorsqu'un utilisateur demande une page, le serveur envoie le fichier HTML correspondant.
  - Chaque action qui nécessite une nouvelle page force un rafraîchissement complet.
- **Caractéristiques :**
  - Simple à créer et à héberger
  - Peu interactif (pas de mise à jour dynamique)
- **Exemples :**
  - Blog personnel avec quelques pages
  - Portfolio ou CV en ligne
  - Site vitrine d'une entreprise locale

Différence entre Front-End et Back-End

Formateur : AARAB HAFID

Aspect	Front-End	Back-End
Définition	Partie visible par l'utilisateur	Partie serveur qui gère les données
Technologies	HTML, CSS, JavaScript	Node.js, Python, PHP, Java, bases de données
Rôle	Créer l'interface utilisateur (boutons, menus, pages)	Fournir et gérer les données (stockage, calcul, sécurité)
Exemple	L'utilisateur parcourt les produits d'un site e-commerce	Le serveur enregistre les commandes et met à jour le stock

Résumé :

- Le **Front-End** gère l'affichage et l'expérience utilisateur.
- Le **Back-End** gère les données, la logique métier et la communication avec la base de données.

Présentation des Frameworks Front-End

- **Définition** : Un **framework** est un ensemble d'outils et de bibliothèques prêts à l'emploi pour faciliter le développement web.
- **Pourquoi utiliser un framework ?**
  - **Gain de temps** : pas besoin de tout coder soi-même

- **Architecture claire** : séparation des parties techniques
- **Maintenance simplifiée** : l'évolution est souvent gérée par la communauté ou l'organisme fondateur

Types de Frameworks

1. **Back-End** : Node.js, Laravel, Django
2. **Front-End applicatif** : React, Angular, Vue.js
3. **Front-End présentation** : Bootstrap…

Exemple :

- Site vitrine simple → HTML/CSS suffisent
- Application interactive → React ou Angular pour gérer les interactions

Concepts de base d'une application web monopage (SPA)

Notion d'application web monopage (SPA)

- **SPA** = Single Page Application

  Charge une seule page HTML et met à jour dynamiquement le contenu via JavaScript, sans recharger toute la page.

- **Exemples :**
  - **Gmail** : pas besoin de recharger la page pour lire un nouvel email
  - **Facebook** : le fil d'actualité se met à jour en continu

- **Trello** : les cartes se déplacent sans rechargement complet

SPA vs MPA (Multi-Page Application)

Aspect	SPA (Single Page Application)	MPA (Multi-Page Application)
Pages	Une seule page	Plusieurs pages HTML
Navigation	Dynamique, contenu mis à jour sans rechargement complet	Chaque action charge une nouvelle page
Performance	Rapide après le chargement initial	Plus lente car chaque page est rechargée
Complexité	Plus complexe à développer	Plus simple à développer
Exemples	Gmail, Trello, Facebook	Amazon, Wikipédia, sites vitrines classiques

Exemple :

- **SPA** : Cliquer sur un onglet “Messages” ne recharge pas la page, juste la section affichée.
- **MPA** : Cliquer sur “Messages” ouvre une nouvelle page HTML avec tout le contenu à recharger.

Fonctionnement d'une SPA

1. L'utilisateur charge la page pour la première fois → HTML + CSS + JS sont chargés.
2. Les interactions (clics, formulaires…) sont gérées sans recharger la page.
3. Les données sont récupérées du serveur via API ou AJAX.
4. L'affichage est mis à jour dynamiquement grâce à JavaScript.

Exemple :

- Dans Gmail, ouvrir un mail ne recharge pas la page entière, le contenu du mail est affiché dynamiquement.

Avantages et inconvénients d'une SPA

Avantages :

- Expérience utilisateur fluide et rapide
- Moins de charge sur le serveur après le chargement initial
- Idéal pour les applications web interactives

Inconvénients :

- Temps de chargement initial parfois long
- SEO plus compliqué (référencement Google)
- Débogage parfois plus complexe

Concept de Virtual DOM (DOM Virtuel)

Définition :

Le **DOM (Document Object Model)** est la structure de la page HTML dans le navigateur.
Lorsque JavaScript modifie un élément (ex : changer un texte, une couleur…), **le navigateur doit mettre à jour le DOM réel**, ce qui peut être **lent** si la page contient beaucoup d'éléments.

Pour améliorer les performances, **React n'agit pas directement sur le DOM réel**, mais utilise **un DOM virtuel (Virtual DOM)**.

DOM classique vs Virtual DOM

Aspect	DOM classique	Virtual DOM (utilisé par React)
Où se font les modifications ?	Directement dans la page réelle	Dans une copie en mémoire (virtuelle)
Réaction à un changement	Le navigateur met à jour toute la partie impactée, parfois plus que nécessaire	React compare l'ancienne et la nouvelle version du Virtual DOM (*diffing*), puis met à jour **uniquement ce qui a changé** dans le DOM réel
Performance	Peut devenir lent si beaucoup de modifications	Très rapide même avec beaucoup de changements

Illustration :

DOM classique : “Tu repeins tout le mur même s’il n’y a qu’une petite tache à corriger.”
Virtual DOM : “Tu compares l’ancien et le nouveau mur, et tu repeins **seulement la tache différente**.”

Exemple :

Imagine un compteur qui augmente à chaque clic :

- En **JavaScript classique**, tu changes directement le texte du , ce qui force le navigateur à recalculer l’affichage.
- En **React**, la valeur change dans le Virtual DOM → React détecte que **seul ce nombre a changé** → met à jour uniquement **cet élément précis**, sans toucher le reste.

Avantages du DOM virtuel

- Mises à jour plus rapides
- Interface fluide comme une application mobile
- Moins de travail pour le navigateur
- Base technique parfaite pour les SPA modernes (React, Vue, etc.)

Petite introduction à React

1. Définition

- **React** est une librairie JavaScript développée par Facebook pour créer des interfaces utilisateur interactives.
- Particulièrement adaptée aux SPA.
- Permet de créer des composants réutilisables et organisés.

2. Concepts clés

JSX vs JavaScript classique

Qu'est-ce que JSX ?

- **Définition :** JSX (JavaScript XML) est une **syntaxe qui permet d'écrire du HTML directement dans du JavaScript**.
- **Pourquoi l'utiliser ?**
  - Facilite la lecture et l'écriture de l'interface utilisateur.
  - Permet de **composer des composants React** avec un code clair et proche du HTML.

Comparaison avec JavaScript classique

Aspect	JavaScript classique	JSX
Écriture d'éléments HTML	document.createElement('div') et appendChild()	Hello World
Lisibilité	Peu lisible pour de grandes interfaces	Lisible comme du HTML normal
Interpolation de variables	Utilisation de **+** ou **innerHTML**	Utilisation de {variable}

Exemple	javascript
	let name = "Ali";
	let div = document.createElement('div');
	div.innerHTML = "Bonjour " + name;
	document.body.appendChild(div);
		javascript
		let name = "Ali";
		 Bonjour {name}

En Résumé : JSX est juste une **façon plus simple et lisible de créer des éléments HTML dans JavaScript**, mais le navigateur ne comprend pas JSX directement → React le transforme en JavaScript classique avant exécution.

ES6 vs JavaScript classique

Qu'est-ce que ES6 ?

- **Définition :** ES6 (ECMAScript 2015) est une version moderne de JavaScript avec de nouvelles fonctionnalités pour coder **plus proprement et efficacement**.
- **Pourquoi c'est important pour React ?**
  - Facilite la gestion des variables, des fonctions et des modules.
  - Permet d'écrire du code **plus lisible et maintenable**, surtout pour les grandes applications.

Comparaison avec JavaScript classique

Fonctionnalité	JavaScript classique	ES6 / moderne
Déclaration de variable	var x = 10;	let x = 10; const y = 20;
Fonctions	function add(a,b){ return a+b; }	const add = (a,b) => a+b;
Concaténation de texte	"Bonjour " + name + " !"	`Bonjour ${name} !`
Modules	Pas natif, besoin de scripts externes	import React from 'react'; export default App;

Fonctionnalité	JavaScript classique	ES6 / moderne
Destructuring	Pas disponible, accès via indices ou propriétés	const {nom, age} = personne; const [a,b] = tableau;

Exemple :

Avec JavaScript classique :

```
var nom = "ali";
var age = 25;
console.log("Bonjour " + nom + ", tu as " + age + " ans");
```

Avec ES6 :

```
const nom = "Ali";
const age = 25;
console.log(`Bonjour ${nom}, tu as ${age} ans`);
```

Résumé :

- **let / const** → gestion plus sécurisée des variables
- **Fonctions fléchées** → syntaxe courte et pratique
- **Template literals** → concaténation plus lisible

- **Destructuring (déstructuration)** → accès plus simple aux objets et tableaux
- **Modules** → organisation du code facilitée

3. Pourquoi JSX + ES6 ensemble ?

- **JSX** permet d'écrire du HTML dans le code JavaScript.
- **ES6** rend ce code **plus propre, modulable et lisible**.
- Ensemble, ils permettent de créer des **interfaces interactives complexes** de manière simple et maintenable.

3. Structure d'un projet React

mon-projet-react/

— node_modules/ # Bibliothèques et dépendances

— public/ # index.html et fichiers statiques

— src/ # Code source

— components/ # Petits blocs réutilisables (boutons, formulaires…)

— pages/ # Pages principales

— App.js # Composant principal qui assemble l'application

— index.js # Point d'entrée qui connecte React au HTML

— styles/ # Fichiers CSS

— package.json # Dépendances et scripts de projet

— README.md # Documentation du projet

Rôle de chaque partie

- **node_modules/** : contient toutes les bibliothèques nécessaires
- **public/** : fichier HTML de base (index.html)
- **src/** : code écrit par le développeur
  - **components/** : blocs réutilisables (ex: boutons, formulaires)
  - **pages/** : pages complètes
  - **App.js** : assemble les composants et pages
  - **index.js** : connecte React au HTML
  - **styles/** : CSS pour l'application
- **package.json** : liste des bibliothèques et scripts
- **README.md** : documentation du projet

4. Points importants à retenir

- **React** est une librairie pour interfaces utilisateur

- Utilise **JSX** pour mélanger HTML et JavaScript
- S’appuie sur **ES6** pour coder proprement et efficacement
- La **structure** d’un projet **React** facilite **maintenance**, **organisation** et **réutilisation**